

```
In [ ]: Algoritmos de Optimización  Actividad Guiada 2 (AG2)  
  
Nombre: Jorge Ramilo Plata  
URL Repositorio Github:  
https://github.com/RamiPla/03MIAR\_PRACTICAS/blob/main/Algoritmos  
\_AG2\_Jorge%20Ramilo%20Plata.ipynb
```

A) EJECUTAR UNICAMENTE EL COMANDO

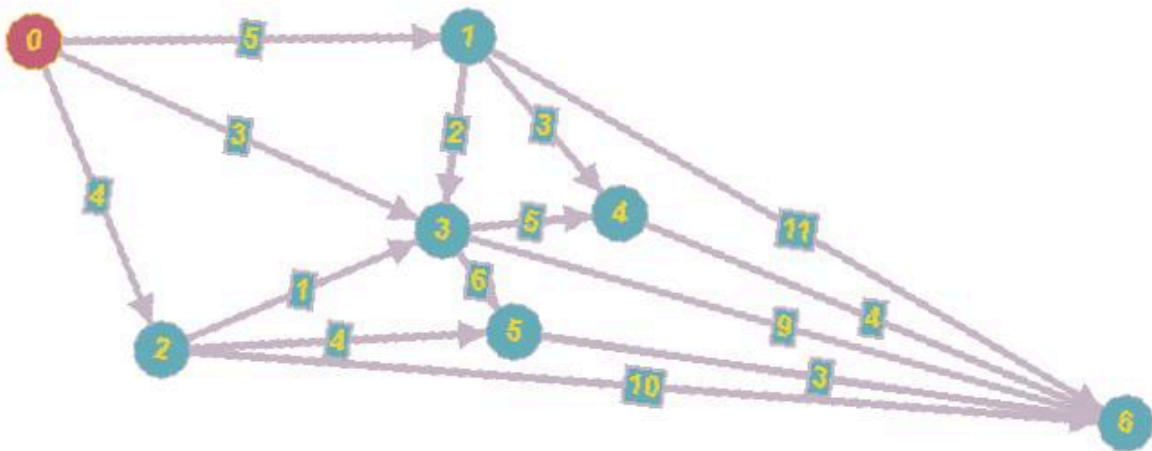
```
In [ ]: import math
```

1. Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: “en una secuencia optima de decisiones, toda sub-secuencia también es óptima” (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.

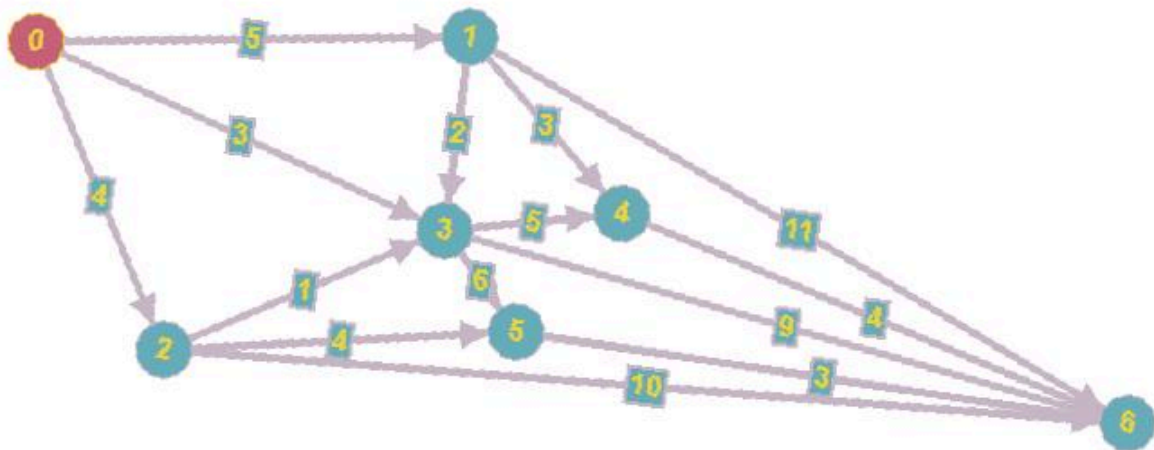


Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: “en una secuencia óptima de decisiones, toda sub-secuencia también es óptima” (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



- Consideramos una tabla $TARIFAS(i,j)$ para almacenar todos los precios que nos ofrecen los embarcaderos.
- Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
In [2]: #Viaje por el rio - Programación dinámica
#####
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999],    #desde nodo 0
[999,0,999,2,3,999,11],    #desde nodo 1
[999,999, 0,1,999,4,10],    #desde nodo 2
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS
```

```
Out[2]: [[0, 5, 4, 3, inf, 999, 999],
[999, 0, 999, 2, 3, 999, 11],
[999, 999, 0, 1, 999, 4, 10],
[999, 999, 999, 0, 5, 6, 9],
[999, 999, 999, 999, 0, 999, 4],
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
In [3]: #Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un
nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a
otro
#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(N)]    #n x n
    RUTA = [ [""]*N for i in range(N)]

    #Se recorren todos los nodos con dos bucles(origen - destino)
    # para ir construyendo la matriz de PRECIOS
    for i in range(N-1):
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k
            PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
```

```
In [4]: PRECIOS, RUTA = Precios(TARIFAS)
        #print(PRECIOS[0][6])
```

```
print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])
```

```
print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])
```

```
PRECIOS
[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
```

```
RUTA
['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
```

```
In [5]: #Calculo de la ruta usando la matriz RUTA
def calcular_ruta(RUTA, desde, hasta):
    if desde == RUTA[desde][hasta]:
        #if desde == hasta:
            #print("Ir a :" + str(desde))
            return desde
    else:
        return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) +
        ',' + str(RUTA[desde][hasta])

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)
```

La ruta es:

```
Out[5]: '0,2,5'
```

2. Problema de Asignación de Tarea - Ramificación y Poda

```
In [13]: #Asignacion de tareas - Ramificación y Poda
#####
#####
#      T A R E A
#      A
#      G
#      E
#      N
#      T
#      E

COSTES=[ [11,12,18,40],
          [14,15,13,22],
          [11,17,19,23],
          [17,14,20,28]]
```

```
In [14]: #Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        VALOR += COSTES[S[i]][i]
    return VALOR

valor((3,2,  ),COSTES)
```

Out[14]: 34

```

In [8]: #Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al
# agente 1

def CI(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):
        VALOR += min( [ COSTES[j][i] for j in range(len(S),
len(COSTES)) ] )
    return VALOR

def CS(S,COSTES):
    VALOR = 0
    #Valores establecidos
    for i in range(len(S)):
        VALOR += COSTES[i][S[i]]

    #Estimacion
    for i in range( len(S), len(COSTES) ):
        VALOR += max( [ COSTES[j][i] for j in range(len(S),
len(COSTES)) ] )
    return VALOR

CI((0,1),COSTES)

```

Out[8]: 68

```

In [15]: #Genera tantos hijos como como posibilidades haya para la
#siguiente elemento de la tupla
#(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N ):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS

```

```

In [16]: crear_hijos((0,) , 4)

```

Out[16]: [{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

```

In [12]: def ramificacion_y_poda(COSTES):
#Construccion iterativa de soluciones(arbol). En cada etapa
asignamos un agente(ramas).
#Nodos del grafo { s:(1,2),CI:3,CS:5 }
#print(COSTES)
DIMENSION = len(COSTES)
MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
CotaSup = valor(MEJOR_SOLUCION,COSTES)
#print("Cota Superior:", CotaSup)

NODOS=[]
NODOS.append({'s':(), 'ci':CI((),COSTES) } )

iteracion = 0

while( len(NODOS) > 0):
    iteracion +=1

    nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]
    ['s']
    #print("Nodo prometedor:", nodo_prometedor)

    #Ramificacion
    #Se generan los hijos
    HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in
crear_hijos(nodo_prometedor, DIMENSION) ]

    #Revisamos la cota superior y nos quedamos con la mejor
solucion si llegamos a una solucion final
    NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION
]
    if len(NODO_FINAL) >0:
        #print("\n*****Soluciones:", [x for x in HIJOS if
len(x['s']) == DIMENSION ] )
        if NODO_FINAL[0]['ci'] < CotaSup:
            CotaSup = NODO_FINAL[0]['ci']
            MEJOR_SOLUCION = NODO_FINAL

    #Poda
    HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]

    #Añadimos los hijos
    NODOS.extend(HIJOS)

    #Eliminamos el nodo ramificado
    NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor
]

    print("La solucion final es:" ,MEJOR_SOLUCION , " en " ,
iteracion , " iteraciones" , " para dimension: " ,DIMENSION )

ramificacion_y_poda(COSTES)

```

La solucion final es: [{ 's': (1, 2, 0, 3), 'ci': 64}] en 10 iteraciones para dimension: 4

3. Descenso de gradiente

```
In [17]: import math                #Funciones matematicas
import matplotlib.pyplot as plt    #Generacion de gráficos (otra
opcion seaborn)
import numpy as np                 #Tratamiento matriz N-
dimensionales y otras (fundamental!)
import scipy as sc

import random
```

Vamos a buscar el minimo de la funcion paraboloides : $f(x) = x^2 + y^2$

Obviamente se encuentra en $(x,y)=(0,0)$ pero probaremos como llegamos a él a través del descenso del gradiente.

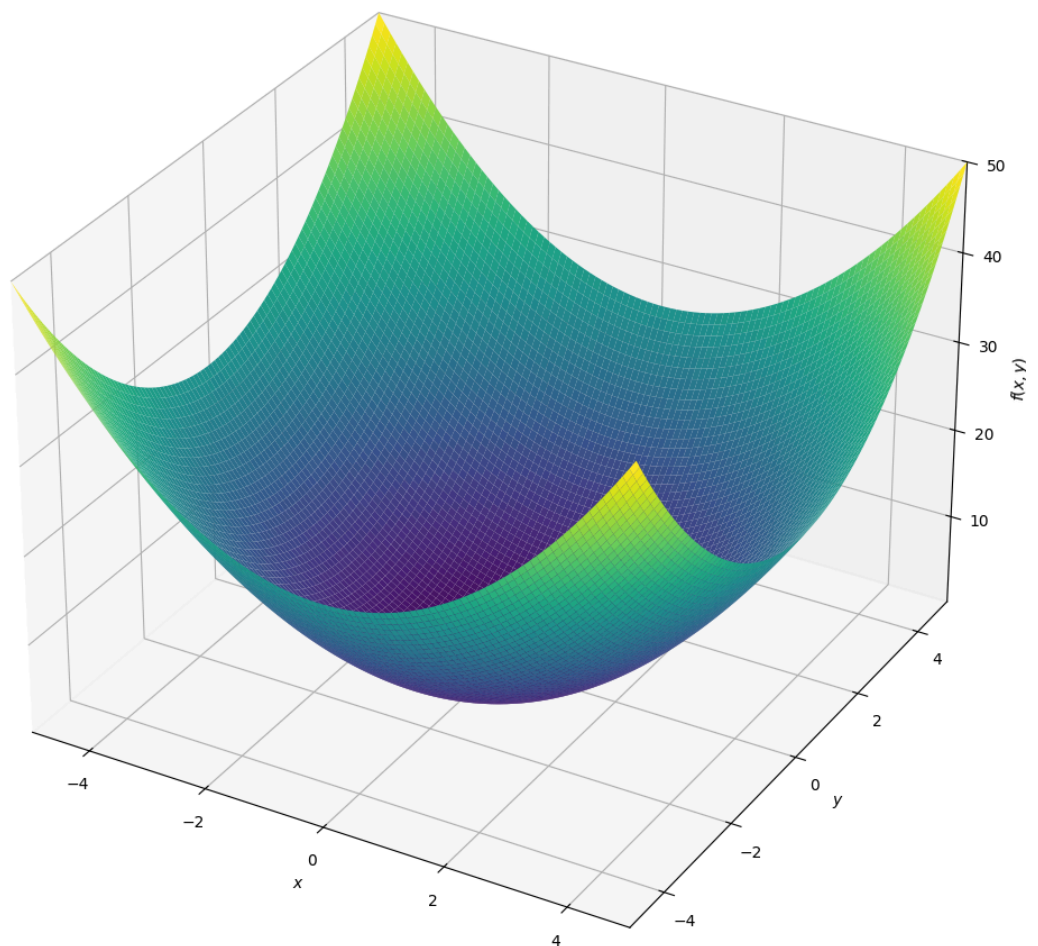
```
In [18]: #Definimos la funcion
#Paraboloides
f = lambda X: X[0]**2 + X[1]**2    #Funcion
df = lambda X: [2*X[0] , 2*X[1]]  #Gradiente

df([1,2])
```

```
Out[18]: [2, 4]
```

```
In [20]: from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
plot3d(x**2 + y**2,
      (x,-5,5),(y,-5,5),
      title='x**2 + y**2',
      size=(10,10))
```

$x^2 + y^2$



```
Out[20]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x22e7f5f20d0>
```

```
In [23]: #Prepara los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=5.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

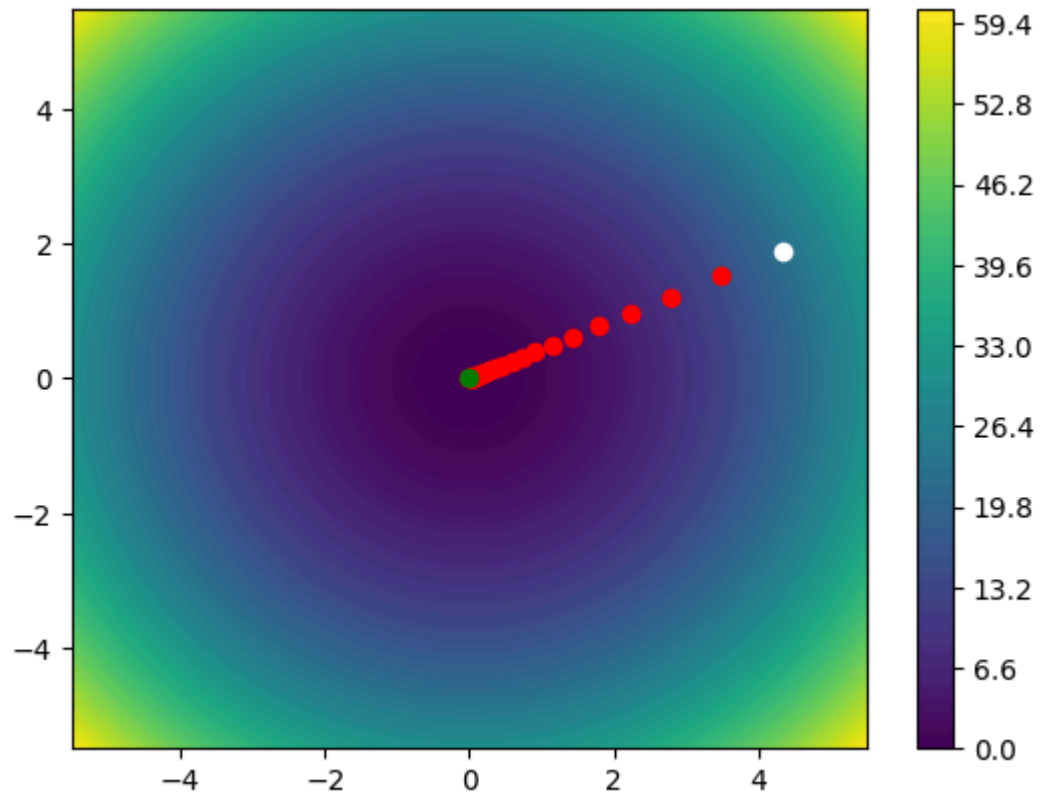
#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-5,5 ),random.uniform(-5,5 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a
medida que nos acercamos.
TA=.1

#Iteraciones:50
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

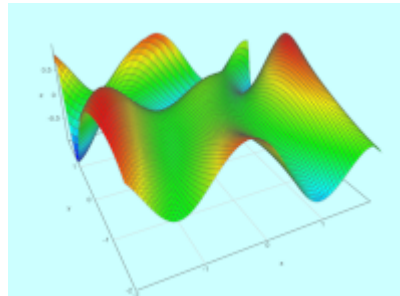
#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))
```



Solucion: [6.194585338352547e-05, 2.7095976241096985e-05]
4.571480679871326e-09

¿Te atreves a optimizar la función?:

$$f(x) = \sin\left(\frac{1}{2}x^2 - \frac{1}{4}y^2 + 3\right) \cos(2x + 1 - e^y)$$



```
In [25]: #Definimos la funcion
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3)
* math.cos(2*X[0] + 1 - math.exp(X[1]))
```

In []:

B) MEJORAS EN EL ALGORITMO

```
In [30]: import itertools
import math
import random
import time

import matplotlib.pyplot as plt
import numpy as np

# La semilla permite repetir los mismos ejemplos.
random.seed(7)
```

1. Programación Dinámica. Viaje por el río

```
In [31]: INFINITO = float("inf")

TARIFAS = [
    [0, 5, 4, 3, INFINITO, INFINITO, INFINITO],
    [INFINITO, 0, INFINITO, 2, 3, INFINITO, 11],
    [INFINITO, INFINITO, 0, 1, INFINITO, 4, 10],
    [INFINITO, INFINITO, INFINITO, 0, 5, 6, 9],
    [INFINITO, INFINITO, INFINITO, INFINITO, 0, INFINITO, 4],
    [INFINITO, INFINITO, INFINITO, INFINITO, INFINITO, 0, 3],
    [INFINITO, INFINITO, INFINITO, INFINITO, INFINITO, INFINITO,
0],
]

print("Número de embarcaderos:", len(TARIFAS))
```

Número de embarcaderos: 7

```

In [32]: def calcular_precios(tarifas):
    n = len(tarifas)
    precios = [[INFINITO] * n for _ in range(n)]
    ruta = [[None] * n for _ in range(n)]

    # Viajar de un embarcadero a sí mismo cuesta cero.
    for i in range(n):
        precios[i][i] = 0
        ruta[i][i] = i

    # Se estudian trayectos cada vez más largos.
    for distancia in range(1, n):
        for origen in range(n - distancia):
            destino = origen + distancia

            # Primera opción: viaje directo.
            precios[origen][destino] = tarifas[origen][destino]
            if tarifas[origen][destino] != INFINITO:
                ruta[origen][destino] = origen

            # Segunda opción: pasar por un punto intermedio.
            for intermedio in range(origen + 1, destino):
                nuevo_precio = (
                    precios[origen][intermedio]
                    + tarifas[intermedio][destino]
                )

                if nuevo_precio < precios[origen][destino]:
                    precios[origen][destino] = nuevo_precio
                    ruta[origen][destino] = intermedio

    return precios, ruta

def reconstruir_ruta(ruta, origen, destino):
    if origen == destino:
        return [origen]

    intermedio = ruta[origen][destino]

    if intermedio is None:
        return []

    if intermedio == origen:
        return [origen, destino]

    return reconstruir_ruta(ruta, origen, intermedio) +
[destino]

```

```
In [33]: PRECIOS, RUTA = calcular_precios(TARIFAS)
ruta_0_6 = reconstruir_ruta(RUTA, 0, 6)

print("Matriz de precios mínimos:")
for fila in PRECIOS:
    print(fila)

print("\nPrecio mínimo de 0 a 6:", PRECIOS[0][6])
print("Ruta completa:", ruta_0_6)

# Casos de prueba.
assert PRECIOS[0][6] == 11
assert ruta_0_6 == [0, 2, 5, 6]
assert reconstruir_ruta(RUTA, 3, 3) == [3]
assert reconstruir_ruta(RUTA, 6, 0) == []
```

```
Matriz de precios mínimos:
[0, 5, 4, 3, 8, 8, 11]
[inf, 0, inf, 2, 3, 8, 7]
[inf, inf, 0, 1, 6, 4, 7]
[inf, inf, inf, 0, 5, 6, 9]
[inf, inf, inf, inf, 0, inf, 4]
[inf, inf, inf, inf, inf, 0, 3]
[inf, inf, inf, inf, inf, inf, 0]
```

```
Precio mínimo de 0 a 6: 11
Ruta completa: [0, 2, 5, 6]
```

\$\$ ***** NOTA ***** \$\$

Este algoritmo comienza por los viajes cortos y utiliza sus precios para calcular viajes más largos.

He realizado una corrección, el código inicial muestra 0,2,5, sin el destino 6, y no inicializaba a cero toda la diagonal de la matriz de precios

In []:

2. Problema de Asignación de Tarea - Ramificación y Poda

```
In [36]: COSTES = [  
    [11, 12, 18, 40],  
    [14, 15, 13, 22],  
    [11, 17, 19, 23],  
    [17, 14, 20, 28],  
    ]  
  
def valor_asignacion(solucion, costes):  
    valor_total = 0  
  
    for agente in range(len(solucion)):  
        tarea = solucion[agente]  
        valor_total += costes[agente][tarea]  
  
    return valor_total  
  
print("Coste de la asignación diagonal:", valor_asignacion((0,  
1, 2, 3), COSTES))
```

Coste de la asignación diagonal: 73

```

In [37]: def cota_inferior(solucion, costes):
    n = len(costes)
    valor_estimado = valor_asignacion(solucion, costes)
    tareas_usadas = set(solucion)

    tareas_disponibles = [
        tarea for tarea in range(n)
        if tarea not in tareas_usadas
    ]

    # Se añade el menor coste posible para cada agente pendiente.
    for agente in range(len(solucion), n):
        menor_coste = min(
            costes[agente][tarea]
            for tarea in tareas_disponibles
        )
        valor_estimado += menor_coste

    return valor_estimado

def crear_hijos(solucion, n):
    hijos = []

    for tarea in range(n):
        if tarea not in solucion:
            hijos.append(solucion + (tarea,))

    return hijos

print("Cota de la solución parcial (0, 2):", cota_inferior((0, 2), COSTES))
print("Hijos de (0,):", crear_hijos((0,), 4))

```

Cota de la solución parcial (0, 2): 55
 Hijos de (0,): [(0, 1), (0, 2), (0, 3)]

```

In [38]: def ramificacion_y_poda(costes):
          n = len(costes)

          if n == 0:
              return (), 0, 0, 0

          mejor_solucion = tuple(range(n))
          mejor_coste = valor_asignacion(mejor_solucion, costes)

          # Cada nodo contiene una solución parcial y su cota.
          nodos = [((), cota_inferior((), costes))]
          nodos_explorados = 0
          nodos_podados = 0

          while len(nodos) > 0:
              # Se selecciona el nodo con menor cota.
              nodos.sort(key=lambda nodo: nodo[1])
              solucion_parcial, cota = nodos.pop(0)

              if cota >= mejor_coste:
                  nodos_podados += 1
                  continue

              nodos_explorados += 1

              for hijo in crear_hijos(solucion_parcial, n):
                  cota_hijo = cota_inferior(hijo, costes)

                  if len(hijo) == n:
                      if cota_hijo < mejor_coste:
                          mejor_solucion = hijo
                          mejor_coste = cota_hijo
                      elif cota_hijo <= mejor_coste:
                          nodos.append((hijo, cota_hijo))
                      else:
                          nodos_podados += 1

          return mejor_solucion, mejor_coste, nodos_explorados,
          nodos_podados

solucion_rp, coste_rp, explorados, podados =
ramificacion_y_poda(COSTES)

print("Mejor asignación:", solucion_rp)
print("Coste mínimo:", coste_rp)
print("Nodos explorados:", explorados)
print("Nodos podados:", podados)

assert solucion_rp == (0, 2, 3, 1)
assert coste_rp == 61
assert ramificacion_y_poda([])[:2] == ((), 0)

```

```

Mejor asignación: (0, 2, 3, 1)
Coste mínimo: 61
Nodos explorados: 7
Nodos podados: 11

```

\$\$ ***** NOTA ***** \$\$

El código va creando las posibles asignaciones por niveles. Para cada una, se calcula el coste que ya tenemos y se añade una estimación del coste que tendrían los agentes que faltan. Si ese coste ya es igual o mayor que el de la mejor solución encontrada, se descarta esa opción y no se sigue por esa rama.

Comprobación mediante fuerza bruta

```
In [39]: def fuerza_bruta(costes):
          n = len(costes)

          if n == 0:
              return (), 0

          mejor_solucion = None
          mejor_coste = INFINITO

          for solucion in itertools.permutations(range(n)):
              coste = valor_asignacion(solucion, costes)

              if coste < mejor_coste:
                  mejor_solucion = solucion
                  mejor_coste = coste

          return mejor_solucion, mejor_coste

solucion_fb, coste_fb = fuerza_bruta(COSTES)

print("Fuerza bruta:", solucion_fb, "-", coste_fb)
print("Ramificación y poda:", solucion_rp, "-", coste_rp)

assert coste_fb == coste_rp == 61
```

```
Fuerza bruta: (0, 2, 3, 1) - 61
Ramificación y poda: (0, 2, 3, 1) - 61
```

In []:

3. Descenso de gradiente

```
In [40]: def descenso_gradiente(
    funcion,
    gradiente,
    punto_inicial,
    tasa_aprendizaje=0.1,
    max_iteraciones=500,
    tolerancia=0.000001,
):
    punto = list(punto_inicial)
    recorrido = [punto.copy()]

    for iteracion in range(max_iteraciones):
        grad = gradiente(punto)
        norma = math.sqrt(grad[0] ** 2 + grad[1] ** 2)

        # Se detiene cuando el gradiente es muy pequeño.
        if norma < tolerancia:
            break

        punto[0] = punto[0] - tasa_aprendizaje * grad[0]
        punto[1] = punto[1] - tasa_aprendizaje * grad[1]
        recorrido.append(punto.copy())

    return punto, funcion(punto), recorrido, iteracion

def paraboloid(punto):
    x, y = punto
    return x ** 2 + y ** 2

def gradiente_paraboloid(punto):
    x, y = punto
    return [2 * x, 2 * y]

punto_paraboloid, valor_paraboloid, recorrido_paraboloid,
iteraciones = (
    descenso_gradiente(
        paraboloid,
        gradiente_paraboloid,
        [4, -3],
        tasa_aprendizaje=0.1,
    )
)

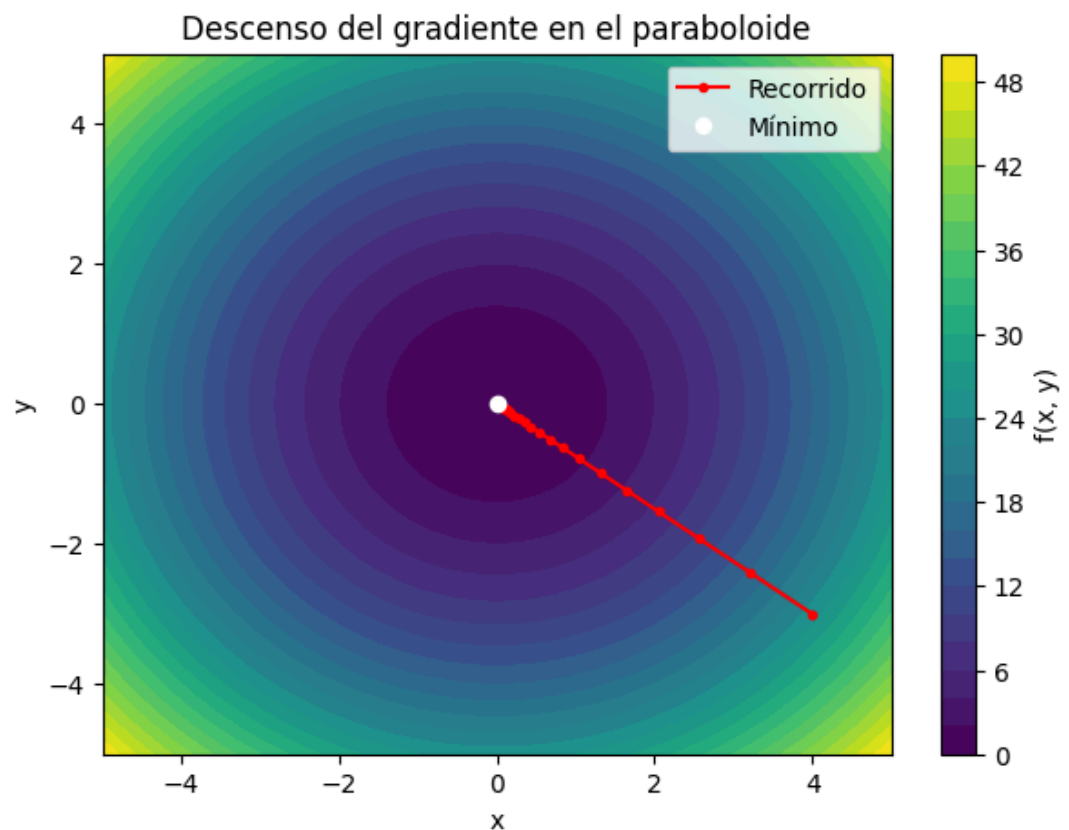
print("Punto final:", punto_paraboloid)
print("Valor final:", valor_paraboloid)
print("Iteraciones:", iteraciones)

assert valor_paraboloid < 0.000001
assert descenso_gradiente(
    paraboloid,
    gradiente_paraboloid,
```

```
[0, 0],  
) [1] == 0
```

Punto final: [3.3699933333938316e-07, -2.5274950000453747e-07]
Valor final: 1.7745086042373237e-13
Iteraciones: 73

```
In [41]: # Se prepara el mapa de niveles.  
valores = np.linspace(-5, 5, 100)  
X, Y = np.meshgrid(valores, valores)  
Z = X ** 2 + Y ** 2  
  
recorrido_array = np.array(recorrido_paraboloide)  
  
plt.figure(figsize=(7, 5))  
plt.contourf(X, Y, Z, levels=30, cmap="viridis")  
plt.colorbar(label="f(x, y)")  
plt.plot(  
    recorrido_array[:, 0],  
    recorrido_array[:, 1],  
    "o-",  
    color="red",  
    markersize=3,  
    label="Recorrido",  
)  
plt.plot(0, 0, "o", color="white", label="Mínimo")  
plt.title("Descenso del gradiente en el paraboloide")  
plt.xlabel("x")  
plt.ylabel("y")  
plt.legend()  
plt.show()
```



\$\$ ***** NOTA ***** \$\$

Partimos del punto (4, -3) y, en cada iteración, actualizamos el punto restando la tasa de aprendizaje multiplicada por el gradiente.

La mejora que he realizado consiste en usar un punto inicial fijo en lugar de uno aleatorio, para que el resultado sea siempre el mismo. Además, he añadido un criterio de parada para finalizar el proceso cuando se cumpla una determinada condición.

In []: